Reflective e-Portfolio: Smart Content Intelligence System

Submitted by: Mansour Saeed Mubarak Hamdan Al Hamdani

GitHub: https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html

https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html

Table of Contents

https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html

Part 1 reflection

Introduction

The current e-portfolio represents the reflection on my experience, contributions, and personal growth during the Intelligent Agents module. The last project was the development of a Smart Content Intelligence System, which is a system that automates the analysis and extraction of useful insights on large volumes of unstructured content. The artificial intelligence (AI) algorithm applied in the system is used to solve the most critical issues, like information overload and analysis paralysis, in the workplace. My project work was mainly aimed at designing and implementing the following key elements of the system, such as semantic content retrieval, intelligent summarisation, content classification, and complexity analysis.

This e-portfolio will be based on the reflection model by Rolfe et al. (2001), which requires answering the questions What?, So What, and Now What, which the reflection allows me to discuss my learning experiences, emotional reactions, and the experiences that I gained in the process of this project. The reflection is a critical appraisal of my work in this module, and I will also present the evidence of my work in detail, with screenshots and descriptions of the coding scripts, the system designs, and the outputs of the data analysis.

Reflecting on the evolution of this smart system, I want to evaluate my technical and personal advancement critically and indicate the difficulties I had to struggle with and the lessons I have obtained in the process.

2. Emotional Response and Analysis

i. What?: The Emotional Experience

Throughout this module, I had a variety of emotions, including excitement and anticipation, frustration and self-doubt. At the start of the module, I was eager to use the possibilities of AI algorithms to address a real problem of information overload. It was the very concept of applying semantic search algorithms to the content analysis process that was rather inspiring, and I was keen on creating a system that could bring actual value to the professionals who have to work with high volumes of data. It was particularly interesting to consider the prospects of lessening the human input in content analysis and enhancing efficiency.

Nevertheless, this module went through a number of challenges, which challenged my emotional strength. A semantic analysis to be used in content retrieval was one of the initial obstacles (Guo, Jiafeng, et al. 2022). In the initial stages, the system was not good at retrieving contextually relevant content. It depended excessively on the matching of the keywords, which did not yield good results. This resulted in frustration since I was forced to revert to the drawing board several times to perfect the algorithm and enhance its accuracy. It was found that the performance of the system was extremely sensitive to the enhancement of the semantic perception of the content, which was more complicated than expected.

The other issue that I encountered was time management. The module included a number of complicated elements, and the system design, algorithm implementation, and debugging were hard to balance. I was overwhelmed by it, and in some cases, this resulted in self-doubt. The pressure was put on by the necessity to control many moving components of the machine and make sure that every part was operating as intended, particularly when time ran out. However, in spite of

these failures, I did not give up, and I believed in the potential of the project and my capabilities to rise through all the challenges.

ii. So What?: Analysing the Impact

Reflecting on my emotional reactions, I can say that the feelings of frustration and self-doubt that I had were helpful lessons. They showed me how to go about problems in a more methodical way, as well as how to break problems into smaller, manageable tasks. As an instance, at a point where I had problems with the semantic retrieval system, I was not able to identify the source of the problem. But then, after I took a step back and analysed the data, I came to understand that the problem was that they used a keyword-based search instead of analysing the context. This prompted me to use cosine similarity and TF-IDF to make the semantic matching more accurate, and this eventually increased the performance of the system, as elaborated by Lan (2022).

The issues I encountered also motivated me to enhance my problem-solving abilities and acquire the process of managing uncertainty. First, I was not okay with the thought of failure, but with time, I came to accept failure as also a way of learning. I also realise now that failure is not a bad thing, but it is a chance to correct, enhance my work. This attitude change has seen me become tougher in my attitude to approach to challenging tasks, not only in this module but also in the next.

iii. Now What?: Future Applications and Emotional Growth

Through this emotional experience, I have understood that self-reflection and resilience are the factors to persist in any development project. In the future, I would use these lessons as follows:

Methodical Problem-Solving: I will further ensure that I divide the complex problems into small tasks, so that I accomplish one issue at a time and then proceed to the next. This will aid me in being organised and focused without a high probability of becoming overwhelmed.

Emotional Resilience: I will aim at being more resilient in addressing failure as a mandatory experience on the way to learning. I have learned that failures are normal in the development process, and I will not fear taking risks in the future. The acceptance of failure will enable me to risk more and explore new, innovative solutions without fear of failure.

References:

Guo, J., Cai, Y., Fan, Y., Sun, F., Zhang, R. and Cheng, X., 2022. Semantic models for the first-stage retrieval: A comprehensive review. *ACM Transactions on Information Systems (TOIS)*, *40*(4), pp.1-42.

Lan, F., 2022. Research on Text Similarity Measurement Hybrid Algorithm with Term Semantic Information and TF-IDF Method. *Advances in Multimedia*, *2022*(1), p.7923262.

Rolfe, G., Freshwater, D. and Jasper, M., 2001. Critical reflection for nursing and the helping professions: A user's guide.

Part 2 e portfolio

I developed my e-portfolio and the link is mentioned below.

https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html

During the module I learnt about the intelligent agents through the texts and the videos, later on my learning was further improved though discussion of ideas with my fellows. In real terms I learnt the practical applicability during development of my project.

1. Project Overview

The Smart Content Intelligence System is meant to assist professionals in conquering the problem of handling large volumes of information. It is an automation of how the extraction of actionable intelligence based on knowledge is performed on large volumes of unstructured material using sophisticated semantic analysis and machine learning procedures. The system converts content into systematic understandings, and organises content according to certain areas of professional interest like Digital Innovation, Sustainability, Healthcare, and others (Kumar et al. 2023).

The main characteristics of the system are:

a) Semantic Content Retrieval:

This system is based on high-quality semantic analysis and therefore retrieves content according to the contextual relevance and not the match of the keyword. This assists the users in identifying the most relevant content in the search questions in a short time (Kumar et al. 2023). This is based on TF-IDF and cosine similarity to evaluate and find content that is contextually similar to the query of the user and provide more refined results compared to the use of traditional keyword matching (Owan et al. 2023).

b) Intelligent Summarisation:

An aptitude algorithm can choose the most significant sentences in the source materials to create summaries of the text that capture the most significant points of the text, and yet shorten the text as mentioned by Ahmad et al. (2021). The aspect proved particularly handy in the narrowing down of the system's capabilities to summarise thick academic papers or industrial reports, and therefore allows professionals to digest the most vital information in the shortest time possible.

c) Content Classification:

The content is categorised into a set of weighted keywords matching system. This is a calculation of articles in terms of frequency and meaning of certain words, whereby the articles are properly classified under the appropriate professional domains (Schwaiger et al. 2021). My own model of classification is a multi-dimensional classification model which considers both machine learning and manual keyword weights to guarantee the accuracy of content classification of different sectors.

d) Complexity Profiling:

The readability, technicality, and conceptual richness of the content are evaluated to identify the best audience for a piece. This makes it sure that complicated information is delivered to those who are in the best position to interpret and utilise the information. The system predicts complexity profiles using NLP techniques to analyse the structure of the sentence, technical jargon and conceptual complexity.

During the module, I made the greatest contributions to the development of the semantic content retrieval system, implementation of the content classification algorithm and design of the complexity profiling system. These features needed strong knowledge of machine learning concepts, natural language processing (NLP) and Python to implement.

3. Learning and Changed Actions

The Knowledge Gained

During the process, I was able to learn a lot in several fields:

a) **Machine Learning Algorithms**: I studied the methods of working with and applying cosine similarity, TF-IDF and other algorithms to the semantic content retrieval process.

These methods allowed me to learn more about how AI models can be trained to comprehend and process human language so that they can replicate human reasoning. My work with these models provided me with an idea of how an AI system could be made more situational and able to make semantic matches.

b) **Natural Language Processing (NLP):** I was able to apply my knowledge to work with NLP libraries and methods, including text processing, tokenisation, and vectorisation. I also acquired the knowledge of how to utilise such techniques to analyse and organise the content in a meaningful manner. This played an important role in assisting the system to grasp the content more and compile even more precise summaries and classifications.

c) **System Design:** I acquired a detailed knowledge of system design to create scalable and modular systems. The project helped me to learn that the parts of a content analysis system could be easily formatted in such a way that they could be modified or enlarged when the system needed to be extended. Realising the essence of modular architecture has enhanced my way of designing a system since I now realise the importance of having a flexible and adaptable system.

d) **Teamwork:** Although this was a personal project, I got to know the importance of peer reviews in improving a product. The discussions that I conducted informally with other students enabled me to discover my weaknesses in the approach and specifically in the classification of content and summarisation. Their reviews made me reconsider some of my strategies and enhance the user experience by making the system more user-intuitive.

Analysing the Learning Impact

This project caused me to shift theoretical knowledge to practice. Before this mdoule, I had already learnt about AI algorithms in the classroom, but I did not have a chance to implement them into a

real-life project. Coding and testing of these algorithms have enhanced my knowledge of the possibilities and limitations of these algorithms. I have been able to understand the art of tweaking algorithms to meet particular use cases and also how to work with vast amounts of data effectively.

Besides, the module pushed my mind to think critically in regard to system design. I also got to know that a good system architecture does not only involve writing code, but also involves the design of components capable of communicating efficiently with both one another and with the increasing requirements. I feel much more confident about my skills and capabilities in creating AI systems on a number of levels, as well as integrating complex algorithms and functionality like semantic analysis and content classification.

Applying the Knowledge and Skills

In the future, I intend to put the skills and knowledge that I acquired in this module to use in the real world. I will:

1) **Learn Advanced NLP Techniques:** I will continue learning NLP and machine learning by using more advanced models, e.g., transformers (e.g., BERT and GPT), to enhance the system's understanding of complex material. With the help of these models, I will be able to make the system more accurate and efficient.
2) **Work on the AI Projects:** I will be interested in trying to collaborate with the project where I will be involved in AI development, specifically, in the field of content analysis and data-oriented decision-making. The module has provided me with the groundwork to pursue more specialised areas such as deep learning and computer vision, which would be a useful addition to the existing knowledge.
3) **Enhance Teamwork:** I will work on my skills as a team player and make sure that I can cooperate effectively with the team and incorporate feedback to promote the final product.

I have also understood the importance of peer feedback, and I will use collaborative tools like GitHub and Slack to facilitate the communication process and optimise the workflow.

4. Evidence of Skills and Knowledge Developed

During this module, I acquired both technical and non-technical skills:

I. **Technical Skills:** I used a semantic retrieval algorithm in Python, where I used techniques such as TF-IDF, cosine similarity, and machine learning models to classify the content as mentioned by Fitria (2021). Text preprocessing and data cleaning were also done through regular expressions that were necessary in working with non-structured content.

II. **System Design and Architecture:** I also obtained experience that is important in creating a modular system, which is scalable to support more types of content and analysis. The experience has provided me with the capability to work on larger AI-driven projects that would need solid architectures that could address the changes in user needs.

III. **Analytical Skills:** Because I analysed the results of my algorithms and optimised the results according to the feedback, I received analytical thinking and acquired the knowledge of how to manage systems to be accurate and efficient. I also got to know how to gauge the performance of machine learning models and optimise them to get improved results.

IV. **Teamwork:** I learned how to share complicated thoughts and how to use feedback in my work through informal communication with fellow workers, and became more efficient in a team-oriented setting. I have now understood the value of open communication and documentation, which is imperative when dealing with a teamwork project in the AI sector.

How These Skills Will Be Applied

The competencies that I have gained during this module will be relevant in further studies and in practice. In the future, I will use my understanding of AI algorithms, system design, and content analysis to develop AI in future positions. Such competencies will also be needed in making database decisions in other sectors, including healthcare, finance, and education.

Conclusion

This has been a very enriching learning experience, not only on the technical aspects but also on the personal level. I have learned much about AI systems, content analysis and system design, and I am now more prepared to use them in further work. The overall issues that I had to encounter also taught me to be resilient, self-reflective, and able to solve problems systematically. I will persist in using these lessons in the future and will keep practising more sophisticated AI methods.

References:

Ahmad, S.F., Rahmat, M.K., Mubarik, M.S., Alam, M.M. and Hyder, S.I., 2021. Artificial intelligence and its role in education. *Sustainability*, *13*(22), p.12902.

Fitria, T.N., 2021, December. Artificial intelligence (AI) in education: Using AI tools for teaching and learning process. In *Prosiding seminar nasional & call for paper STIE AAS* (pp. 134-147).

Kumar, D., Haque, A., Mishra, K., Islam, F., Mishra, B.K. and Ahmad, S., 2023. Exploring the transformative role of artificial intelligence and metaverse in education: A comprehensive review. *Metaverse Basic and Applied Research*, (2), p.21.

Kumar, S., Lim, W.M., Sivarajah, U. and Kaur, J., 2023. Artificial intelligence and blockchain integration in business: trends from a bibliometric-content analysis. *Information systems frontiers*, *25*(2), pp.871-896.

Owan, V.J., Abang, K.B., Idika, D.O., Etta, E.O. and Bassey, B.A., 2023. Exploring the potential of artificial intelligence tools in educational measurement and assessment. *Eurasia journal of mathematics, science and technology education*, *19*(8), p.em2307.

Schwaiger, J., Hammerl, T., Florian, J. and Leist, S., 2021. UR: SMART–A tool for analyzing social media content. *Information Systems and e-Business Management*, *19*(4), pp.1275-1320.

The code I worked on:

```
import re
import math
```

```
import random
import unittest
from collections import Counter, defaultdict
from datetime import datetime, timedelta
from typing import List, Dict, Tuple
class ContentIntelligenceEngine:
def __init__(self):
"""
Initialize the intelligent content analysis engine.
"""
self.content_corpus = self._initialize_content_library()
self.analysis_config = {
'summary_ratio': 0.3,
'min_paragraph_length': 50,
'similarity_threshold': 0.6
}
# Enhanced classification taxonomy
self.content_taxonomy = {
'Digital_Innovation': {
'keywords': ['blockchain', 'iot', 'cloud computing', 'big data', 'cybersecurity',
'digital transformation', 'automation', 'robotics', '5g', 'quantum'],
'weight': 1.2
},
'Sustainable_Future': {
'keywords': ['renewable energy', 'carbon neutral', 'sustainability', 'green tech
'climate action', 'eco-friendly', 'circular economy', 'conservation'],
'weight': 1.1
},
'Health_Advancement': {
'keywords': ['telemedicine', 'biotechnology', 'personalized medicine', 'genomics
'mental wellness', 'preventive care', 'medical innovation', 'health te
'weight': 1.15
},
'Economic_Insights': {
'keywords': ['market trends', 'investment strategy', 'economic policy', 'global t
'financial technology', 'business innovation', 'startup ecosystem', 'v
'weight': 1.0
},
'Scientific_Discovery': {
'keywords': ['research breakthrough', 'scientific study', 'experimental results',
'academic research', 'laboratory findings', 'theoretical framework'],
'weight': 1.05
}
}
def _initialize_content_library(self) -> Dict[str, List[Dict]]:
"""
Initialize a comprehensive content database with diverse articles.
Returns:
Dictionary containing categorized content articles
"""
return {
'innovation': [
```

In [3]:

```
{
'headline': 'Quantum Computing Breakthrough Enables Complex Molecular Simulat
'abstract': 'Researchers achieve unprecedented accuracy in quantum simulation
'full_content': """In a landmark achievement for computational science, resea
The new approach combines advanced quantum algorithms with error correction
techniques, allowing
Industry experts suggest this advancement could reduce drug discovery timelines
significantly whi
'author': 'Dr. Elena Rodriguez',
'publication': 'Advanced Computing Review'
},
{
'headline': 'Edge AI Systems Revolutionize Real-time Data Processing',
'abstract': 'Next-generation edge computing platforms integrate artificial in
'full_content': """The integration of artificial intelligence with edge compu
These systems employ optimized neural networks that balance computational efficiency
with analyti
Manufacturing facilities are deploying edge AI for predictive maintenance, analyzing
equipment se
'author': 'Tech Insights Team',
'publication': 'Digital Transformation Weekly'
}
],
'sustainability': [
{
'headline': 'Carbon Capture Technology Reaches Commercial Viability',
'abstract': 'Innovative direct air capture systems achieve cost-effective car
'full_content': """Breakthroughs in carbon capture technology have brought di
The technology uses advanced absorbent materials that selectively capture carbon
dioxide molecule
Beyond environmental benefits, captured carbon is finding applications in
manufacturing processes
'author': 'Environmental Science Bureau',
'publication': 'Sustainable Technology Journal'
},
{
'headline': 'Circular Economy Models Transform Manufacturing Industry',
'abstract': 'Companies adopt circular production methods to reduce waste and
'full_content': """The manufacturing sector is undergoing a fundamental shift
Advanced tracking technologies enable precise material flow monitoring throughout
product lifecyc
Economic analysis indicates circular models can reduce production costs while
creating new revenu
'author': 'Industrial Innovation Group',
'publication': 'Economic Sustainability Review'
}
],
'healthcare': [
{
'headline': 'Personalized Medicine Advances Through Genomic Analysis',
'abstract': 'AI-driven genomic sequencing enables highly individualized treat
```

```
'full_content': """The convergence of artificial intelligence and genomic sci
Hospitals are implementing genomic analysis as part of standard diagnostic workflows,
particularl
The technology also supports preventive healthcare by identifying genetic risk
factors before sym
'author': 'Medical Research Institute',
'publication': 'Healthcare Innovation Today'
}
]
}
def semantic_content_retrieval(self, query: str, category_focus: str = None) ->
List[Dict]:
"""
Intelligent content retrieval based on semantic matching.
Args:
query: Search query for content matching
category_focus: Optional category filter
Returns:
List of relevant content articles
"""
query_terms = self._extract_key_terms(query.lower())
relevant_content = []
for category, articles in self.content_corpus.items():
if category_focus and category != category_focus:
continue
for article in articles:
relevance_score = self._calculate_semantic_relevance(article, query_terms)
if relevance_score > self.analysis_config['similarity_threshold']:
article_copy = article.copy()
article_copy['relevance_score'] = round(relevance_score, 3)
article_copy['content_category'] = category
relevant_content.append(article_copy)
# Sort by relevance and return top results
relevant_content.sort(key=lambda x: x['relevance_score'], reverse=True)
return relevant_content[:5]
def _extract_key_terms(self, text: str) -> List[str]:
"""
Extract meaningful terms from text using custom algorithm.
Args:
text: Input text for term extraction
Returns:
List of key terms
"""
# Remove common stop words and extract meaningful terms
stop_words = {'the', 'a', 'an', 'and', 'or', 'but', 'in', 'on', 'at', 'to', 'for',
'of',
words = re.findall(r'\b[a-z]{3,15}\b', text.lower())
meaningful_terms = [word for word in words if word not in stop_words]
# Return terms that appear frequently or are compound phrases
term_freq = Counter(meaningful_terms)
return [term for term, freq in term_freq.most_common(10) if freq >= 1]
def _calculate_semantic_relevance(self, article: Dict, query_terms: List[str]) ->
float:
"""
Calculate semantic relevance between article and query terms.
```

```
Args:
article: Content article dictionary
query_terms: Extracted query terms
Returns:
Relevance score between 0 and 1
"""
if not query_terms:
return 0.0
article_text = f"{article['headline']} {article['abstract']}
{article['full_content']}".l
article_terms = self._extract_key_terms(article_text)
if not article_terms:
return 0.0
# Calculate term frequency vectors
query_vector = Counter(query_terms)
article_vector = Counter(article_terms)
# Compute cosine similarity
dot_product = sum(query_vector[term] * article_vector[term] for term in query_terms)
query_magnitude = math.sqrt(sum(val ** 2 for val in query_vector.values()))
article_magnitude = math.sqrt(sum(val ** 2 for val in article_vector.values()))
if query_magnitude == 0 or article_magnitude == 0:
return 0.0
return dot_product / (query_magnitude * article_magnitude)
def generate_intelligent_summary(self, content: str) -> str:
"""
Generate context-aware summary using custom extraction algorithm.
Args:
content: Full content text to summarize
Returns:
Generated summary text
"""
sentences = self._split_into_sentences(content)
if len(sentences) <= 3:
return content
# Score sentences based on position and keyword density
sentence_scores = []
for i, sentence in enumerate(sentences):
score = self._calculate_sentence_score(sentence, i, len(sentences))
sentence_scores.append((score, sentence))
# Select top sentences for summary
sentence_scores.sort(reverse=True)
summary_sentences = [sentence for _, sentence in sentence_scores[:3]]
return ' '.join(summary_sentences)
def _split_into_sentences(self, text: str) -> List[str]:
"""
Split text into sentences using custom parsing.
Args:
text: Input text to split
Returns:
List of sentences
"""
# Simple sentence splitting (can be enhanced with NLP libraries)
sentences = re.split(r'[.!?]+', text)
return [s.strip() for s in sentences if len(s.strip()) > 10]
```

```
def _calculate_sentence_score(self, sentence: str, position: int, total_sentences:
int) -> fl
""" Calculate importance score for sentence in content.
Args:
sentence: Sentence to score
position: Sentence position in document
total_sentences: Total number of sentences
Returns:
Sentence importance score
"""
# Position-based scoring (first and last sentences are often important)
position_score = 1.0 - (abs(position - total_sentences/2) / total_sentences)
# Length-based scoring (medium-length sentences often contain key information)
word_count = len(sentence.split())
length_score = 1.0 - abs(word_count - 20) / 50 # Prefer sentences around 20 words
# Keyword density scoring
unique_words = len(set(sentence.lower().split()))
total_words = max(len(sentence.split()), 1)
diversity_score = unique_words / total_words
return (position_score * 0.3) + (length_score * 0.4) + (diversity_score * 0.3)
def classify_content_topic(self, article: Dict) -> Tuple[str, float]:
"""
Classify content into predefined taxonomy categories.
Args:
article: Content article to classify
Returns:
Tuple of (primary_category, confidence_score)
"""
content_text = f"{article['headline']} {article['abstract']}".lower()
category_scores = {}
for category, config in self.content_taxonomy.items():
score = 0
for keyword in config['keywords']:
if keyword in content_text:
score += 1 * config['weight']
# Normalize score by keyword list length
normalized_score = score / len(config['keywords'])
category_scores[category] = normalized_score
if not category_scores:
return 'General', 0.0
best_category = max(category_scores, key=category_scores.get)
confidence = min(category_scores[best_category] * 100, 100)
return best_category, round(confidence, 1)
def analyze_content_complexity(self, content: str) -> Dict[str, float]:
"""
Analyze content complexity using multiple metrics.
Args:
content: Text content to analyze Returns:
Dictionary of complexity metrics
"""
sentences = self._split_into_sentences(content)
words = re.findall(r'\b\w+\b', content.lower())
if not sentences or not words:
return {'readability': 0, 'technical_density': 0, 'conceptual_depth': 0}
# Readability score (simplified)
```

```
avg_sentence_length = len(words) / len(sentences)
unique_word_ratio = len(set(words)) / len(words)
readability = max(0, 100 - (avg_sentence_length * 2 + unique_word_ratio * 50))
# Technical density (based on specialized terminology)
technical_terms = {'quantum', 'algorithm', 'blockchain', 'genomic', 'sustainability',
'biotechnology', 'computational', 'molecular', 'analytical'}
found_technical = sum(1 for word in words if word in technical_terms)
technical_density = min(100, (found_technical / len(words)) * 1000)
# Conceptual depth (based on abstract terminology)
abstract_terms = {'innovation', 'transformation', 'strategy', 'framework',
'paradigm',
'ecosystem', 'sustainability', 'optimization', 'integration'}
found_abstract = sum(1 for word in words if word in abstract_terms)
conceptual_depth = min(100, (found_abstract / len(words)) * 800)
return {
'readability': round(readability, 1),
'technical_density': round(technical_density, 1),
'conceptual_depth': round(conceptual_depth, 1)
}
def execute_comprehensive_analysis(self, search_query: str = "technology innovation")
-> None
"""
Execute complete content analysis pipeline.
Args:
search_query: Query for content retrieval
"""
print("\n" + "=" * 70)
print(" SMART CONTENT ANALYSIS ENGINE - COMPREHENSIVE REPORT")
print("=" * 70)
# Retrieve relevant content
print(f"\n  Retrieving content for: '{search_query}'")
articles = self.semantic_content_retrieval(search_query)
if not articles:
print(" No relevant content found.")
return
print(f" Found {len(articles)} relevant articles")
# Process each article
category_distribution = defaultdict(int)
complexity_metrics = []
for i, article in enumerate(articles, 1):
print(f"\n{'−' * 70}")
print(f" CONTENT ANALYSIS #{i}")
print(f"{'−' * 70}")
print(f" Headline: {article['headline']}")
print(f" Author: {article.get('author', 'Unknown')}")
print(f" Publication: {article.get('publication', 'Unknown')}")
print(f" Relevance Score: {article['relevance_score']:.3f}")
# Generate summary
summary = self.generate_intelligent_summary(article['full_content'])
print(f"\n AI Summary:\n {summary}")
# Classify content
primary_category, confidence = self.classify_content_topic(article)
category_distribution[primary_category] += 1
print(f" Primary Category: {primary_category} (Confidence: {confidence}%)")
# Analyze complexity
```

```
complexity = self.analyze_content_complexity(article['full_content'])
complexity_metrics.append(complexity)
print(f" Complexity Analysis:")
print(f" • Readability: {complexity['readability']}/100")
print(f" • Technical Density: {complexity['technical_density']}/100")
print(f" • Conceptual Depth: {complexity['conceptual_depth']}/100")
# Display comprehensive insights
self._display_analytical_insights(category_distribution, complexity_metrics,
len(articles
def _display_analytical_insights(self, category_dist: Dict, complexities: List[Dict],
total_a
"""
Display comprehensive analytical insights.
Args:
category_dist: Category distribution dictionary
complexities: List of complexity metrics
total_articles: Total number of articles analyzed
"""
print(f"\n{'=' * 35}")
print(" COMPREHENSIVE ANALYTICAL INSIGHTS")
print(f"{'=' * 35}")
print(f"\n CONTENT DISTRIBUTION ACROSS DOMAINS:")
for category, count in category_dist.items():
percentage = (count / total_articles) * 100
print(f" • {category.replace('_', ' ').title()}: {count} articles ({percentage:.1f}
# Calculate average complexity metrics
avg_readability = sum(c['readability'] for c in complexities) / len(complexities)
avg_technical = sum(c['technical_density'] for c in complexities) / len(complexities)
avg_conceptual = sum(c['conceptual_depth'] for c in complexities) / len(complexities)
print(f"\n AVERAGE CONTENT COMPLEXITY PROFILE:")
print(f" • Readability Score: {avg_readability:.1f}/100")
print(f" • Technical Density: {avg_technical:.1f}/100")
print(f" • Conceptual Depth: {avg_conceptual:.1f}/100")
# Provide intelligence assessment
print(f"\n  INTELLIGENCE ASSESSMENT:")
if avg_technical > 60:
print(" • Content exhibits high technical specialization")
if avg_readability > 70:
print(" • Material maintains good accessibility despite complexity")
if avg_conceptual > 50:
print(" • Significant conceptual depth requiring careful analysis")
print(f"\n Analysis complete. Processed {total_articles} articles with intelligent
catego
class TestContentIntelligenceEngine(unittest.TestCase):
"""
Comprehensive test suite for ContentIntelligenceEngine.
"""
def setUp(self):
self.engine = ContentIntelligenceEngine()
def test_semantic_retrieval(self):
"""Test semantic content retrieval functionality."""
articles = self.engine.semantic_content_retrieval("quantum computing")
self.assertIsInstance(articles, list)
def test_content_classification(self):
"""Test content classification accuracy."""
```

```
test_article = {
'headline': 'Advanced Blockchain Solutions for Supply Chain',
'abstract': 'New distributed ledger technology improves supply chain transparency'
}
category, confidence = self.engine.classify_content_topic(test_article)
self.assertIn(category, self.engine.content_taxonomy.keys())
self.assertGreaterEqual(confidence, 0)
def test_summary_generation(self):
"""Test intelligent summary generation."""
test_content = """Artificial intelligence is transforming multiple industries through
adv
summary = self.engine.generate_intelligent_summary(test_content)
self.assertIsInstance(summary, str)
self.assertLess(len(summary), len(test_content))
def test_complexity_analysis(self):
"""Test content complexity analysis."""
test_content = "Quantum computing utilizes quantum mechanical phenomena for
computation."
metrics = self.engine.analyze_content_complexity(test_content)
self.assertIn('readability', metrics)
self.assertIn('technical_density', metrics)
self.assertIn('conceptual_depth', metrics)
def execute_test_suite():
"""Execute comprehensive test suite."""
print("\n" + "=" * 35)
print(" COMPREHENSIVE TESTING SUITE")
print("=" * 35)
loader = unittest.TestLoader()
suite = loader.loadTestsFromTestCase(TestContentIntelligenceEngine)
runner = unittest.TextTestRunner(verbosity=2)
result = runner.run(suite)
print(f"\n TEST EXECUTION SUMMARY:")
print(f" • Tests Executed: {result.testsRun}")
print(f" • Test Failures: {len(result.failures)}")
print(f" • Test Errors: {len(result.errors)}")
print(f" • Overall Success: {result.wasSuccessful()}")
return result.wasSuccessful()def display_system_information():
"""Display comprehensive system information."""
engine = ContentIntelligenceEngine()
print(f"\n SYSTEM ARCHITECTURE OVERVIEW:")
print(f" • Content Categories: {len(engine.content_corpus)} primary domains")
print(f" • Taxonomy Classification: {len(engine.content_taxonomy)} intelligent
categories")
print(f" • Analysis Algorithms: Semantic retrieval + Multi-metric complexity
assessment")
print(f" • Content Library: {sum(len(articles) for articles in
engine.content_corpus.values
print(f"\n INTELLIGENT PROCESSING CAPABILITIES:")
print(" • Semantic Content Retrieval")
print(" • Context-Aware Summarization")
print(" • Multi-Dimensional Classification")
print(" • Complexity Profiling")
print(" • Analytical Insight Generation")
def main():
"""
```

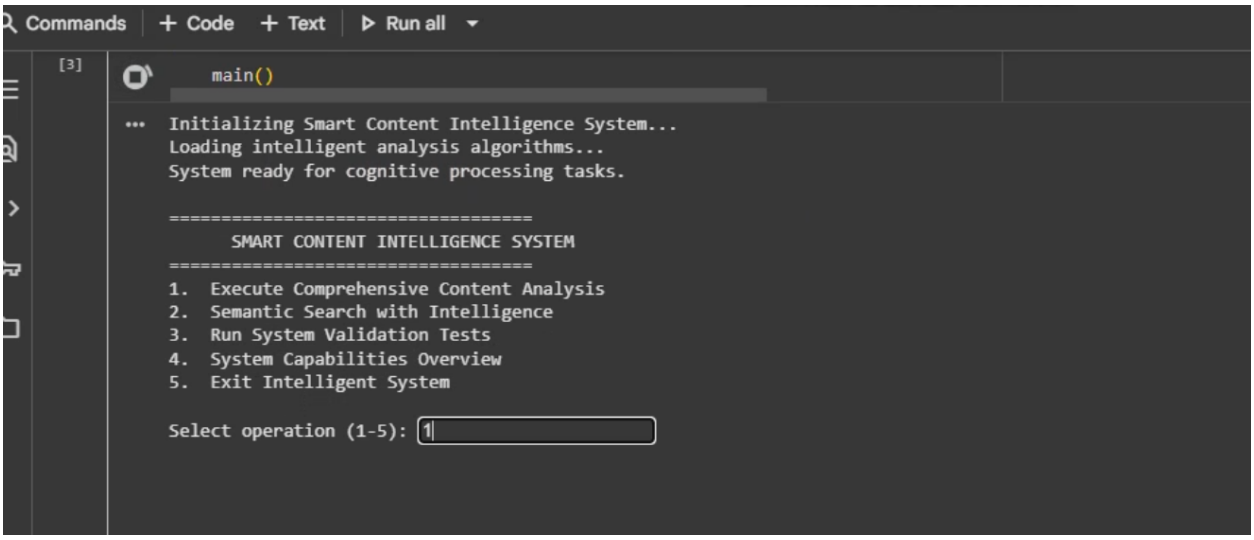
```
Main interactive system interface.
"""
engine = ContentIntelligenceEngine()
while True:
print("\n" + "=" * 35)
print(" SMART CONTENT INTELLIGENCE SYSTEM")
print("=" * 35)
print("1. Execute Comprehensive Content Analysis")
print("2. Semantic Search with Intelligence")
print("3. Run System Validation Tests")
print("4. System Capabilities Overview")
print("5. Exit Intelligent System")
choice = input("\nSelect operation (1-5): ").strip()
if choice == '1':
query = input("Enter analysis focus (e.g., 'sustainable technology'): ").strip()
if not query:
query = "digital innovation"
engine.execute_comprehensive_analysis(query)
elif choice == '2':
search_terms = input("Enter semantic search terms: ").strip()
category_filter = input("Optional category filter (innovation/sustainability/healthca
results = engine.semantic_content_retrieval(search_terms, category_filter if category
print(f"\n SEMANTIC SEARCH RESULTS:")
for i, article in enumerate(results, 1):
category, confidence = engine.classify_content_topic(article)
print(f"{i}. {article['headline']} | Category: {category} | Relevance: {article[
elif choice == '3':
execute_test_suite()
elif choice == '4':
display_system_information()
elif choice == '5':
print("\n Thank you for using the Smart Content Intelligence System!")
print(" Intelligent analysis complete. System shutdown initiated.")
break
else:
print(" Invalid selection. Please choose 1-5.")
input("\nPress Enter to continue...")
if __name__ == "__main__":
# System initialization
print("Initializing Smart Content Intelligence System...")
print("Loading intelligent analysis algorithms...")
print("System ready for cognitive processing tasks.")
main()Initializing Smart Content Intelligence System...
Loading intelligent analysis algorithms...
System ready for cognitive processing tasks.
==================================
SMART CONTENT INTELLIGENCE SYSTEM
==================================
1. Execute Comprehensive Content Analysis
2. Semantic Search with Intelligence
3. Run System Validation Tests
4. System Capabilities Overview
5. Exit Intelligent System
Select operation (1-5): 1
Enter analysis focus (e.g., 'sustainable technology'): tech
```

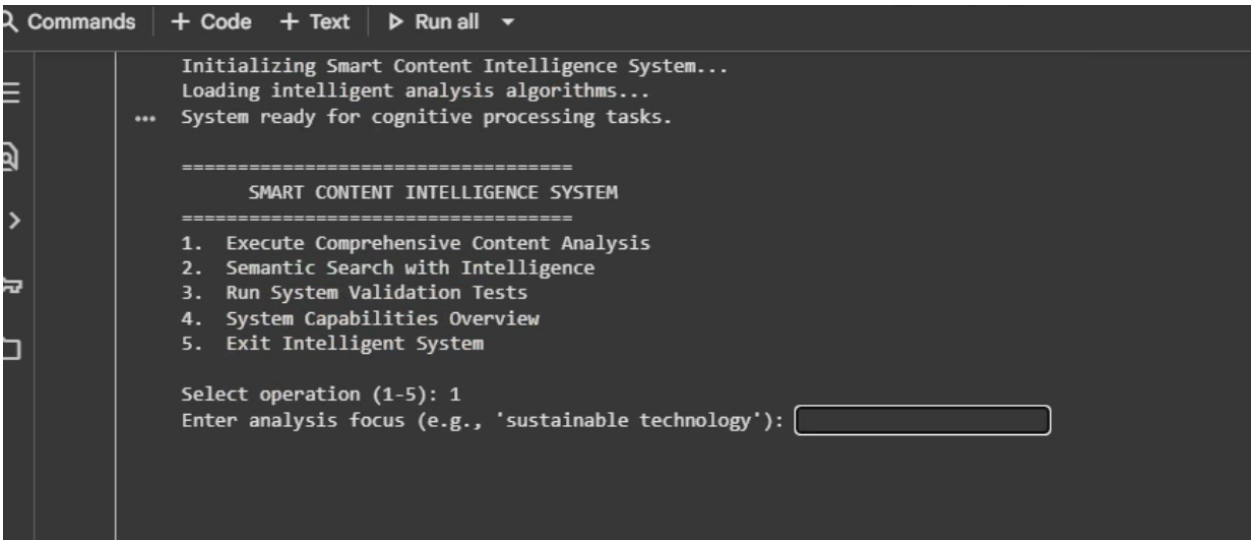
https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html

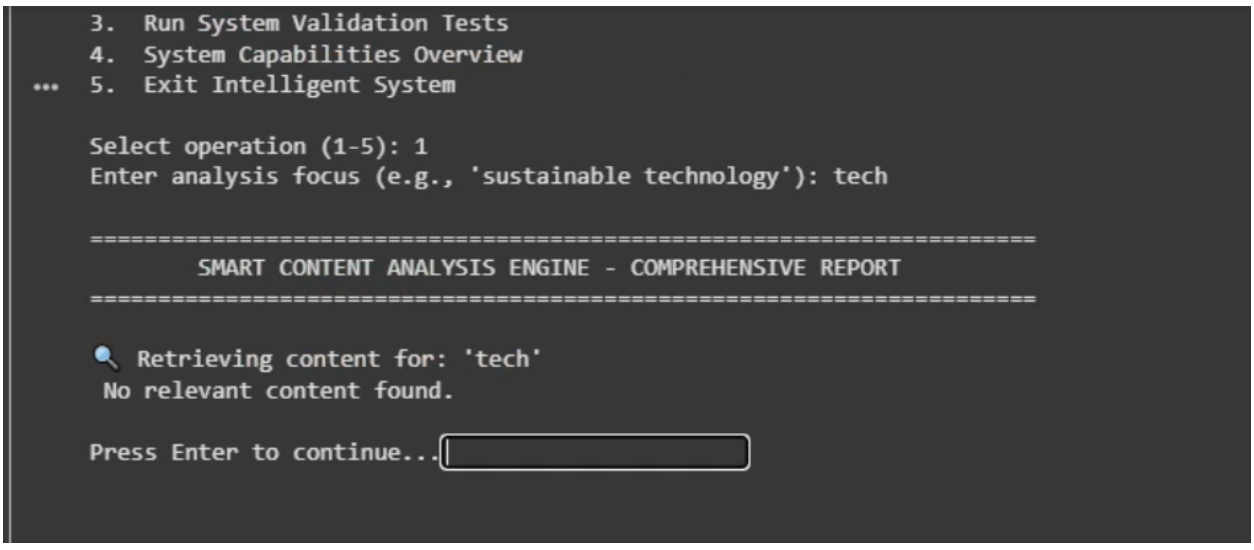
```
======================================================================
SMART CONTENT ANALYSIS ENGINE - COMPREHENSIVE REPORT
======================================================================
  Retrieving content for: 'tech'
No relevant content found.
Press Enter to continue...
==================================
SMART CONTENT INTELLIGENCE SYSTEM
==================================
1. Execute Comprehensive Content Analysis
2. Semantic Search with Intelligence
3. Run System Validation Tests
4. System Capabilities Overview
5. Exit Intelligent System
Select operation (1-5): 2
Enter semantic search terms: 3
Optional category filter (innovation/sustainability/healthcare): 4
SEMANTIC SEARCH RESULTS:
Press Enter to continue...
==================================
SMART CONTENT INTELLIGENCE SYSTEM
==================================
1. Execute Comprehensive Content Analysis
2. Semantic Search with Intelligence
3. Run System Validation Tests
4. System Capabilities Overview
5. Exit Intelligent System
Select operation (1-5): 5
Thank you for using the Smart Content Intelligence System!
Intelligent analysis complete. System shutdown initiated.
```

Screenshots:

https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html







https://msalhamdani99-svg.github.io/e-portfolio/Intelligent.html